Tracking the Perpetrator

A quick look at Pod Evictions

Agenda

- What is eviction and disruption?

- Why pod disruptions are important?

- Pod eviction types

Eviction & Disruption

Eviction is the process of terminating one or more Pods on Nodes.

Disruption is the event of a pod becoming unavailable, regardless of the cause.

Eviction is the **API and mechanism**.

Voluntary disruption → triggered by eviction (drain, autoscaler, rolling update).

Involuntary disruption → caused by unexpected failures (hardware failure, OOMKill, network partition).

Isn’t the Kubernetes eventually consistent, why pod disruptions are important?

It’s important because of the stateful applications! They do might survive graceful shutdowns but...

Pod evictions

- Eviction API
- Node-pressure eviction
- Taint-based eviction
- Kubelet admission
- Pod preemption
- Node deletion

Eviction API

Eviction API provides a safe and controlled way to terminate Pods. Unlike a direct DELETE operation, calling the Eviction API respects any PodDisruptionBudget (PDB) you have configured.

Workload controllers such as ReplicaSets, Deployments, and StatefulSets will delete Pods directly during a rollout, without taking PodDisruptionBudgets (PDBs) into account.

The main consumers of the eviction API are:

- kubectl drain: For gracefully cordoning and draining nodes.
- Cluster Autoscaler: For safely removing pods from nodes that are being scaled down.

37

kubewharf / podseidon

A multi-cluster pod deletion protection webhook with high scalability and disaster tolerance

6 Contributors

Node-pressure Eviction

The kubelet monitors node resources such as memory, disk, and inodes. When these become constrained, it may evict pods to free resources and avoid starvation. In such cases, the kubelet marks the affected pods as Failed and terminates them.

This node-pressure eviction differs from API-initiated evictions. It doesn’t uses the Eviction API and immediately terminate pods based on soft or hard evictions.

Soft Eviction:Triggered when resource usage (e.g., memory, disk, inodes) exceeds a configured threshold for a grace period. Pods are only evicted if the condition persists.

Hard Eviction: Triggered immediately when resource usage crosses a strict limit. Pods are evicted without waiting.

Taint Based Eviction

- The kube-controller-manager monitors heartbeats from each node's Kubelet. If a heartbeat fails, it taints the node with NoExecute.

- The taint manager controller (previously part of the node-lifecycle controller) is responsible for evicting pods from nodes that have the NoExecute taint.

```
→  ~ kubectl get events
LAST SEEN   TYPE     REASON                 OBJECT         MESSAGE
11m         Normal   TaintManagerEviction   pod/busybox    Marking for deletion Pod default/busybox
11m         Normal   Killing                pod/busybox    Stopping container busybox
```

kubernetes/enhancements

#166 Taint Based Eviction

111 comments

davidopp opened on January 20, 2017

Kubernetes v1.34: Decoupled Taint Manager Is Now Stable

By **Baofa Fan (DaoCloud)** | Monday, September 15, 2025

Restarting kubelet do not impact workloads

Kubelet Admission

This type of eviction is uncommon and rarely observed. It occurs because the Kubelet runs certain predicates as admission checks before allowing a Pod onto the node.

Kubelet runs these admission checks not once, but also after a kubelet restart.

```
→  ~ kubectl label node talos-b5h-hvy role=test
node/talos-b5h-hvy labeled
→  ~ kubectl apply -f pod.yaml
pod/busybox created
→  ~ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
busybox   1/1     Running   0          18s
→  ~ kubectl label node talos-b5h-hvy role-
node/talos-b5h-hvy unlabeled
→  ~ talosctl services kubelet restart
NODE           RESPONSE
10.0.153.137   Service "kubelet" restarted
→  ~ kubectl get pods
NAME      READY   STATUS   RESTARTS   AGE
busybox   0/1     Error    0          67s
→  ~ kubectl get pods busybox -o yaml | yq '.status.message'
Pod was rejected: Predicate NodeAffinity failed: node(s) didn't match Pod's node affinity/selector
```

What was the reason for pod to fail?

Because after a restart Kubelet doesn't have any state to load from so it builds it's cache from scratch and at that time it sees all pods as "Add" operation so runs those admission checks all over the pods and in such case kubelet might reject pod(s).

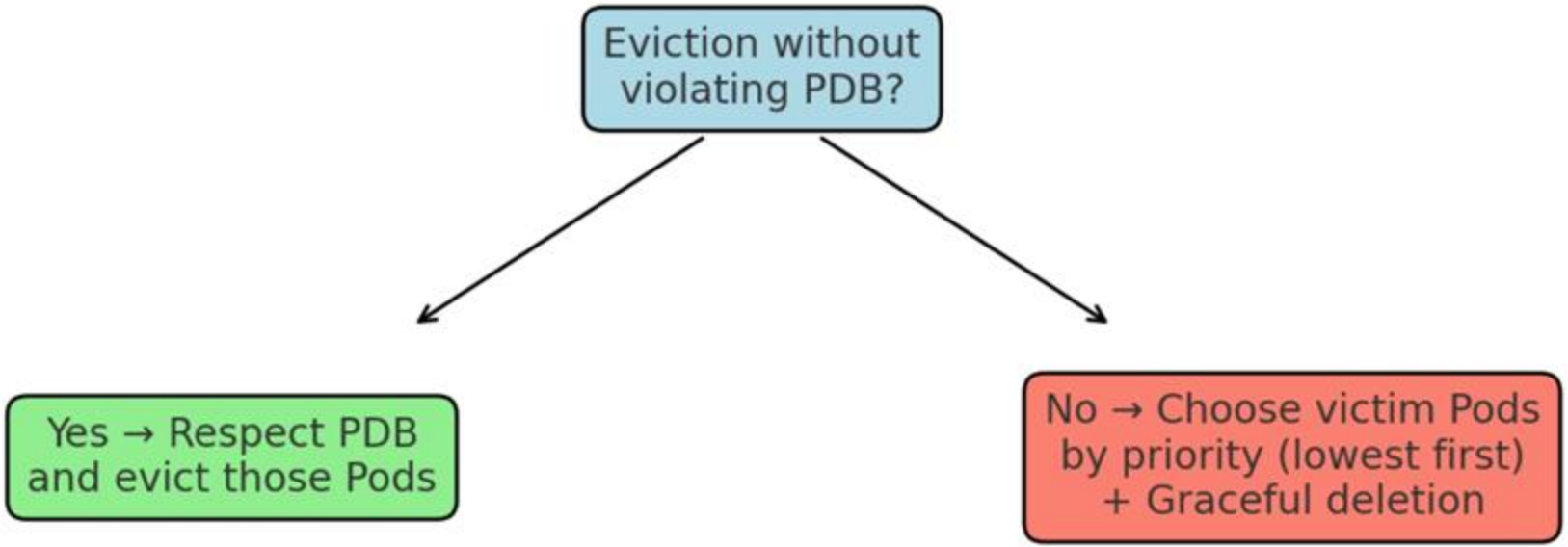
```
                    // After restarting, kubelet will get all existing pods
ugh
                    // ADD as if they are new pods. These pods will then go
ugh the
                    // admission process and *may* be rejected. This can be
lved
                    // once we have checkpointing.
```

Pod Preemption

Beyond node drainers, taint manager, and kubelet, the kube-scheduler can also terminate Pods. When no node fits a pending Pod, the scheduler considers evicting lower-priority Pods to make room.

kube-scheduler rank nodes based by at least PDB violations. If it’s possible it chooses the node which doesn’t violate the PDB if not then it will look for priority of pod(s).

Preemption process



🚀 Method	👤 Eviction Actor	📋 PDB Respected?	🔴 Graceful Termination?
Eviction API	Node drainers, `kubectl drain`	✅ Yes	✅ Yes
Direct Pod Deletion	Workload controllers during rollouts (e.g., ReplicaSet, StatefulSet) or `kubectl delete`	❌ No	✅ Yes
Node Pressure (Soft)	`kubelet`	❌ No	✅ Yes
Node Pressure (Hard)	`kubelet`	❌ No	❌ No
NoExecute Taint	`kube-controller-manager` (taint manager)	❌ No	✅ Yes
Kubelet Admission	`kubelet`	❌ No	❌ No
Priority Preemption	`kube-scheduler`	⚠️ Best Effort	✅ Yes
Node Deletion	`kube-controller-manager` (podgc controller)	❌ No	❌ No

Thanks for listening!

alperenchelik

alperencelik